Homework Solutions

Mohammad Abu Shariha

October 13, 2025

Problem 1. Strongly Connected Components (20 pts)

Graph (i)

1. DFS on G: A systematic traversal (starting from A, exploring neighbors alphabetically) yields the following finishing order, from last to first:

$$\text{Finish order (high to low): } \texttt{E}, \texttt{B}, \texttt{A}, \texttt{C}, \texttt{D}, \texttt{F}, \texttt{H}, \texttt{J}, \texttt{I}, \texttt{G}$$

2. DFS on G^R: We process vertices in decreasing order of finishing times (E, B, A, ...).

- Start DFS from `E`: finds SCC $\{\texttt{E}\}$
- Start DFS from `B`: finds SCC $\{\texttt{B}\}$
- Start DFS from `A`: finds SCC $\{\texttt{A}\}$
- Start DFS from `C`: finds SCC $\{\texttt{C}\}$
- Start DFS from `D`: finds SCC $\{\texttt{D}\}$
- Start DFS from `F`: finds SCC $\{\texttt{F}\}$
- Start DFS from `H`: finds SCC $\{\texttt{H}\}$
- Start DFS from `J`: finds SCC $\{\texttt{J}\}$
- Start DFS from `I`: explores `G`, which explores `I` (already in stack). Finds SCC $\{\texttt{G}, \texttt{I}\}$

Final SCCs found (in order):

$$\{\texttt{E}\}, \{\texttt{B}\}, \{\texttt{A}\}, \{\texttt{C}\}, \{\texttt{D}\}, \{\texttt{F}\}, \{\texttt{H}\}, \{\texttt{J}\}, \{\texttt{G}, \texttt{I}\}$$

Graph (ii)

1. DFS on G: A traversal starting from A yields the finish order:

$$\text{Finish order (high to low): } \texttt{B}, \texttt{C}, \texttt{A}, \texttt{D}, \texttt{E}, \texttt{F}, \texttt{G}, \texttt{H}, \texttt{I}$$

2. DFS on G^R: Processing vertices in this order reveals that there are no cycles between vertices (only a self-loop on H). Therefore, every vertex forms its own SCC. **Final SCCs found (in order):**

$$\{\texttt{B}\}, \{\texttt{C}\}, \{\texttt{A}\}, \{\texttt{D}\}, \{\texttt{E}\}, \{\texttt{F}\}, \{\texttt{G}\}, \{\texttt{H}\}, \{\texttt{I}\}$$

(

b) Which are source and sink SCCs? **Graph (i):**

- **Source SCCs** (no incoming edges in metagraph): {E}, {B}
- **Sink SCCs** (no outgoing edges in metagraph): {J}

Graph (ii):

- **Source SCCs**: {B}
- **Sink SCCs**: {I}

(c) Metagraphs

Each meta-node corresponds to an SCC.

(i) Metagraph for first graph:

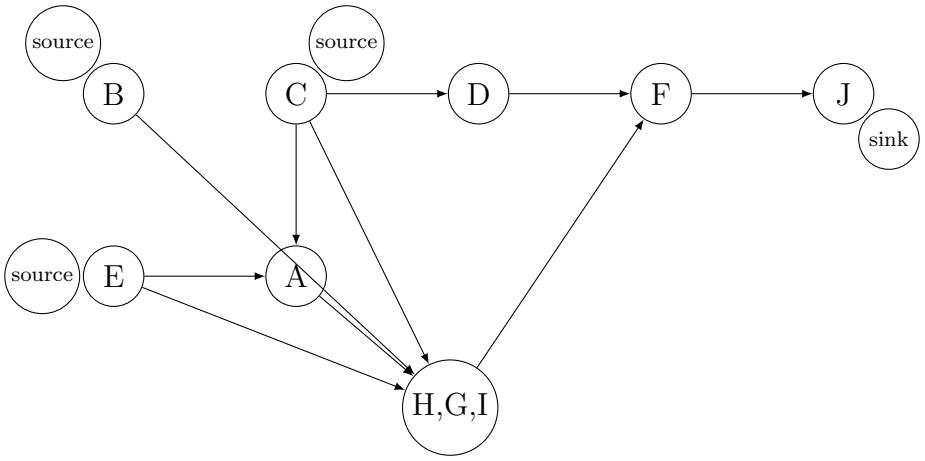


Figure 1: Metagraph for Graph (i)

(ii) Metagraph for second graph:

(d) Minimum edges to make graph strongly connected

The minimum number of edges required is:

$$\max(\#\text{source SCCs}, \#\text{sink SCCs})$$

Graph (i): $\max(3, 1) = 3$ edges needed. **Graph (ii):** $\max(1, 1) = 1$ edge needed.

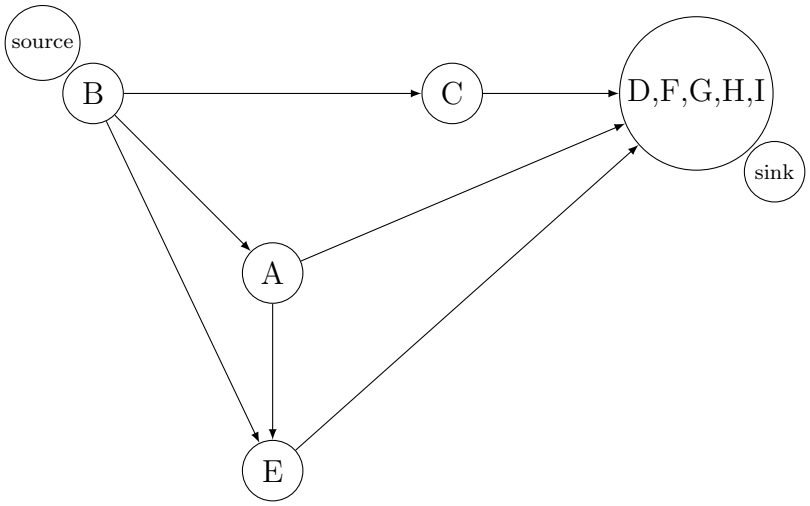


Figure 2: Metagraph for Graph (ii)

Problem 2. Universal Reachability Algorithm (20 pts)

An efficient algorithm to determine if a vertex s exists from which all other vertices are reachable is as follows:

Algorithm:

1. Run Kosaraju's algorithm or Tarjan's algorithm on the graph $G = (V, E)$ to find all of its Strongly Connected Components (SCCs).
2. Construct the component graph, G^{SCC}, where each node represents an SCC from step 1. An edge exists from SCC C_i to C_j in G^{SCC} if there is an edge in G from a vertex in C_i to a vertex in C_j.
3. Count the number of "source" SCCs in G^{SCC}. A source SCC is one with an in-degree of 0.
4. If there is **exactly one** source SCC, then such a vertex s exists (any vertex within that single source SCC will work). Otherwise, no such vertex exists.

Running Time: The running time is dominated by finding the SCCs.

- Step 1 (Kosaraju's algorithm) takes $O(|V| + |E|)$ time.
- Step 2 (Building G^{SCC}) takes $O(|V| + |E|)$ time by iterating through all vertices and edges once.
- Step 3 (Counting source SCCs) takes $O(|V| + |E|)$ time.

The total running time is $\mathbf{O}(|\mathbf{V}| + |\mathbf{E}|)$.

Correctness: The component graph G^{SCC} is a Directed Acyclic Graph (DAG). For a vertex s to reach all other vertices in G, it must belong to an SCC that can reach all other SCCs in G^{SCC}. This is only possible if s is in a source SCC. If there is more than one source SCC, vertices in one source component cannot reach vertices in another, so no single vertex can reach everyone. If there are zero source SCCs, it implies a cycle in the component graph, which is impossible. Therefore, a universal starting vertex s exists if and only if there is exactly one source SCC.

Problem 3. Unique Shortest Path (20 pts)

Idea: A path from s to u is unique if there is exactly one shortest path.

We can detect non-unique shortest paths using Dijkstra's algorithm while tracking path counts.

Algorithm:

1. Run Dijkstra from s.
2. For each vertex v, store both its distance $d[v]$ and number of shortest paths $count[v]$.
3. When relaxing edge (u, v):

$$\text{if } d[v] = d[u] + l(u,v): \quad count[v] += count[u]$$

$$\text{else if } d[v] > d[u] + l(u,v): \quad d[v] = d[u] + l(u,v),\ count[v] = count[u]$$

4. After Dijkstra, set $\texttt{usp}[v] = \text{true}$ iff $\texttt{count}[v] = 1$.

Time complexity: $O((|V| + |E|) \log |V|)$ (same as Dijkstra).

Correctness: We only increment counts when alternative shortest paths exist, so count[v] = 1 ensures uniqueness.

Problem 4. Dijkstra's Algorithm with Negative Edges (20 pts)

Given that only edges leaving s are negative and s has no incoming edges:

Observation: Dijkstra fails when it extracts a vertex whose distance is not final (i.e., when a later relaxation would lower it). However, since only edges leaving s are negative, after the first relaxation step, all remaining edges are nonnegative.

Argument:

- The distance to each neighbor v of s becomes $w(s, v)$, possibly negative.
- All other distances are ∞ initially.
- The algorithm will then extract the neighbor with smallest $d[v]$ first, but no further edge decreases can propagate back to already-visited vertices because there are no negative cycles and no edge enters s.

Conclusion: Dijkstra's algorithm still produces correct results in this restricted case.

Problem 5. BFS on Directed Graphs (20 pts)

(a) Why no BFS forward edges?

In BFS, each edge (u, v) satisfies:

$$\text{level}[v] \leq \text{level}[u] + 1$$

Thus, edges can only connect to nodes at the same level or one level deeper. Forward edges (to non-child descendants) would require $\text{level}[v] > \text{level}[u] + 1$, which never happens. Hence, BFS forward edges are impossible.

(b) Classification Algorithm

Input: Directed graph $G = (V, E)$, BFS tree levels.

1. Run BFS from source s, recording each vertex's `level`.
2. For every edge (u, v):
   - If v is a tree child of u, mark as **Tree edge**.
   - Else if $\text{level}[v] = \text{level}[u]$, mark as **Cross edge**.
   - Else if $\text{level}[v] < \text{level}[u]$, mark as **Back edge**.

Complexity: $O(|V| + |E|)$.

Explanation: Levels uniquely determine edge types in BFS. No forward edges occur because BFS explores all vertices level by level.